

BOT GPT

AI Engineer Case Study · BOT Consulting

NestJS

PostgreSQL + pgvector

Gemini 2.5 Flash

LangChain

Next.js 15

Aman Pawar · 2026

What We Built

A full-stack AI chat platform with two conversation modes:

Open Chat

Streaming conversations with Gemini 2.5 Flash. Sliding window or summarization context strategy per conversation.

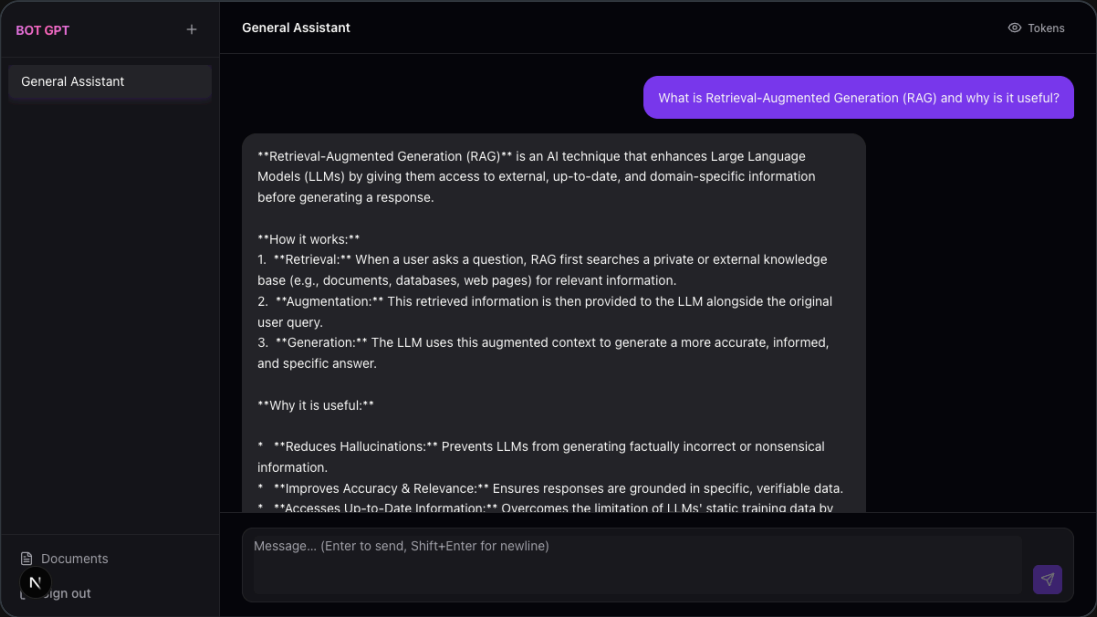
RAG Chat

Upload PDFs → embedded into pgvector → every answer grounded in your documents, not training data.

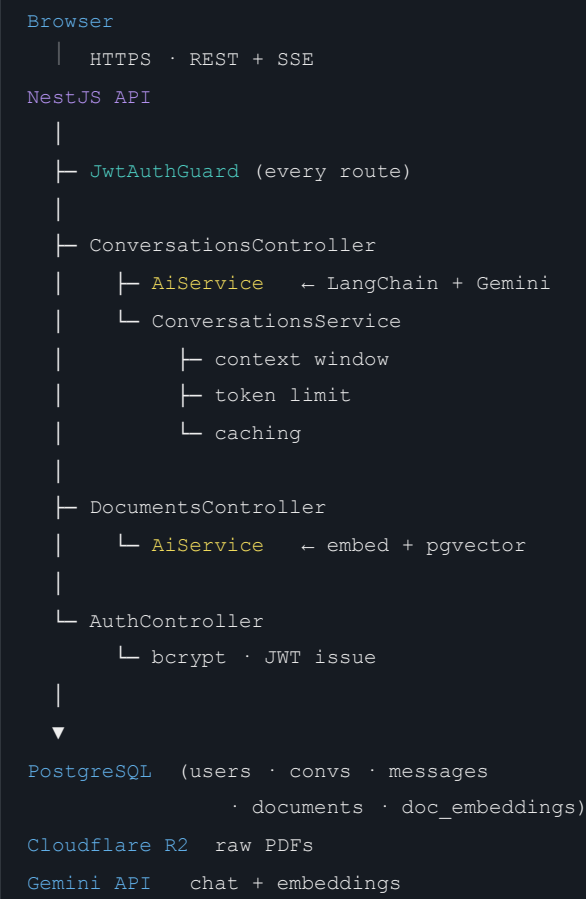
Production features

JWT auth · daily token limits · retry backoff · in-memory caching · multi-stage Docker · GitHub Actions CI

Live app

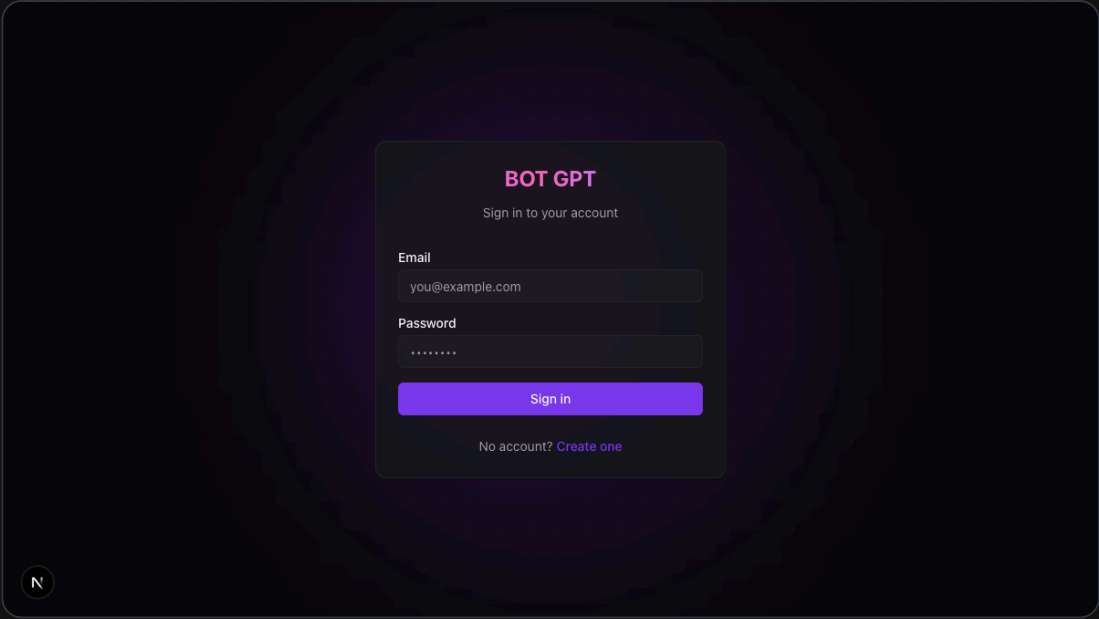
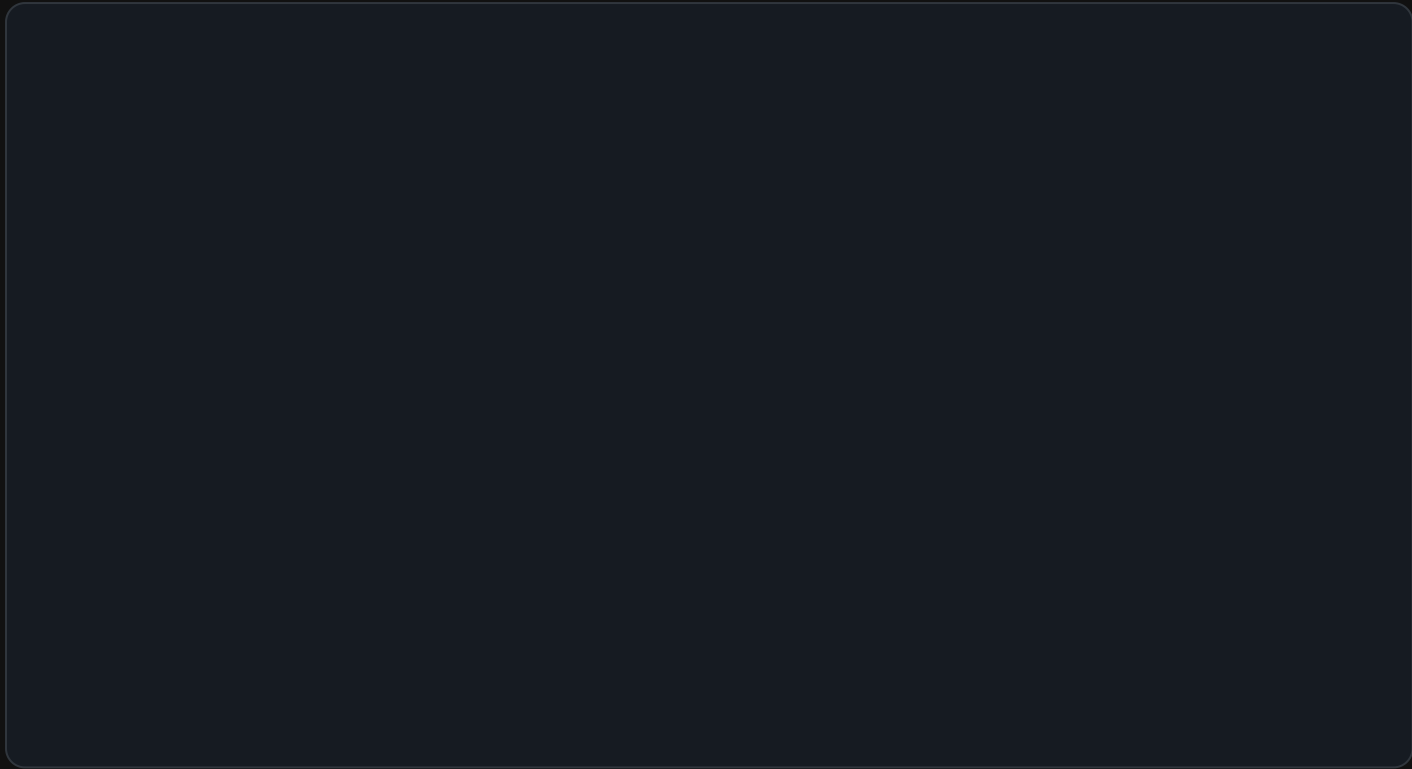


Architecture



- ✓ **Single DB** — pgvector inside Postgres handles both relational data and vector similarity. No Pinecone/Weaviate.
- ✓ **Stateless API** — JWT means no session store. Every instance is identical. Scales horizontally.
- ✓ **SSE not WebSocket** — unidirectional, works over HTTP/1.1, perfect for token streaming.
- ✓ **LangChain** — unified interface for streaming, embeddings, vector store. Avoids hand-rolling SSE parsing.

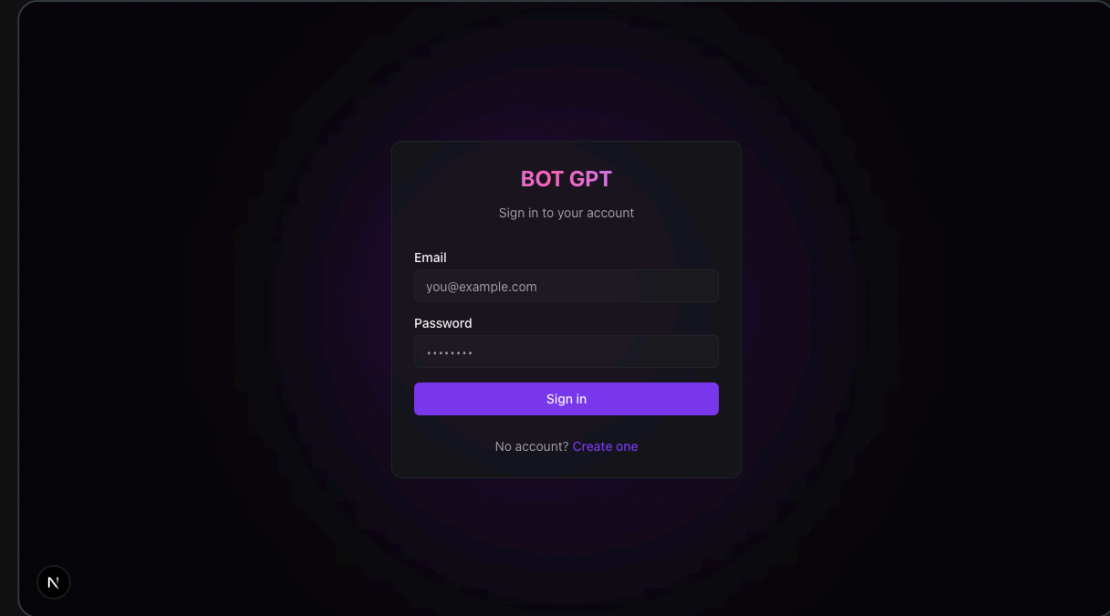
Auth: JWT Flow



Stateless — no session store needed

Auth: JWT Flow

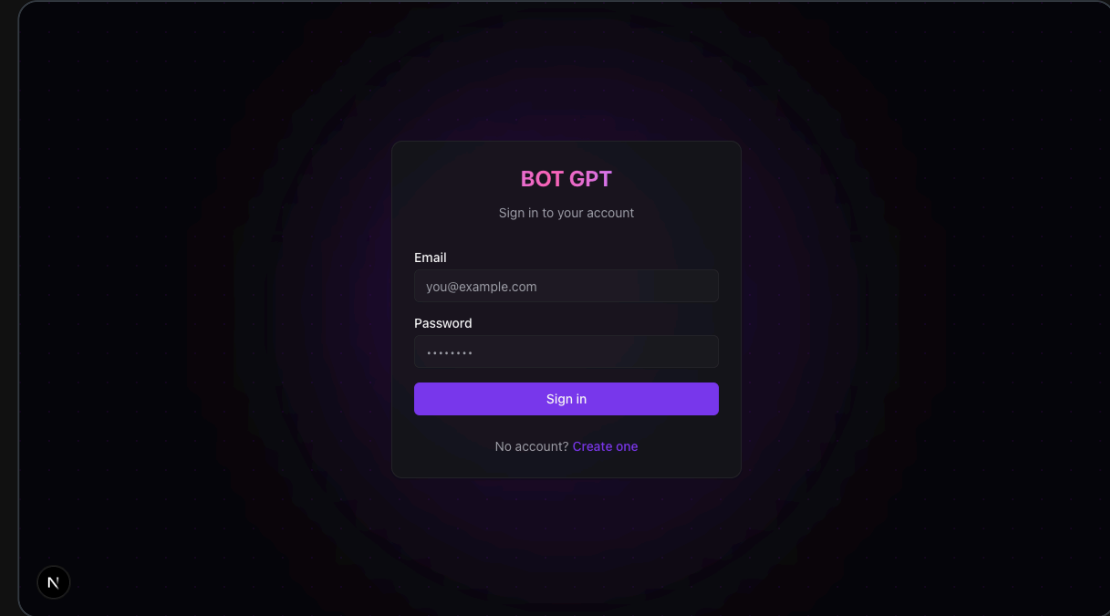
```
POST /auth/register { email, password }
```



Stateless — no session store needed

Auth: JWT Flow

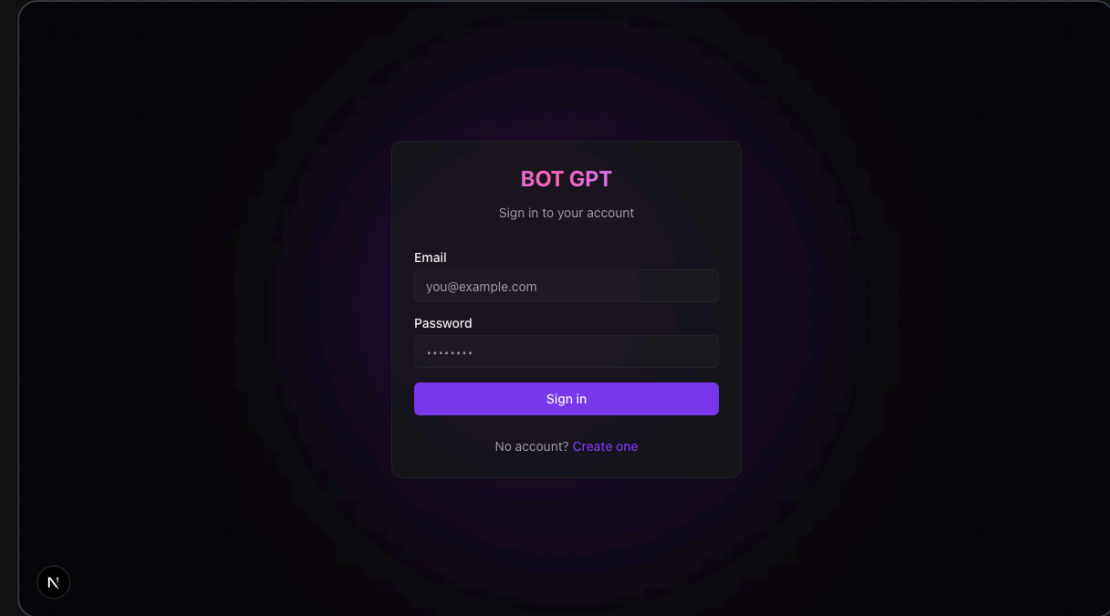
```
POST /auth/register { email, password }  
  └─ bcrypt.hash(password, 10)  
  └─ INSERT INTO users  
    409 ConflictException if email exists
```



Stateless — no session store needed

Auth: JWT Flow

```
POST /auth/register { email, password }
├─ bcrypt.hash(password, 10)
├─ INSERT INTO users
│   409 ConflictException if email exists
└─ jwt.sign({ sub: userId, email })
    → { access_token }
```

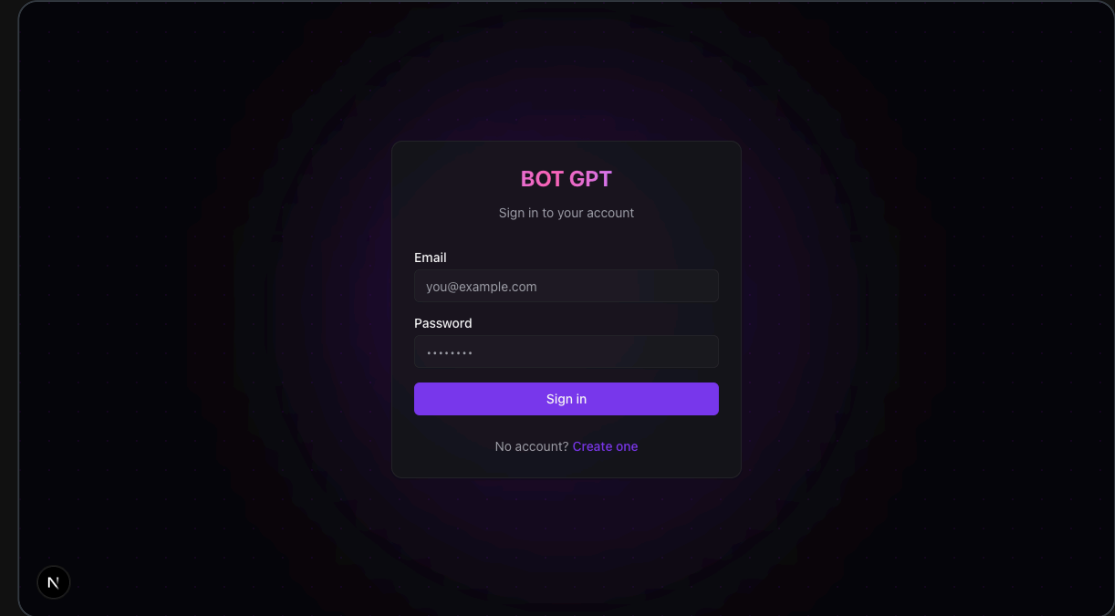


Stateless — no session store needed

Auth: JWT Flow

```
POST /auth/register { email, password }
├─ bcrypt.hash(password, 10)
├─ INSERT INTO users
│   409 ConflictException if email exists
└─ jwt.sign({ sub: userId, email })
    → { access_token }
```

```
POST /auth/login { email, password }
```

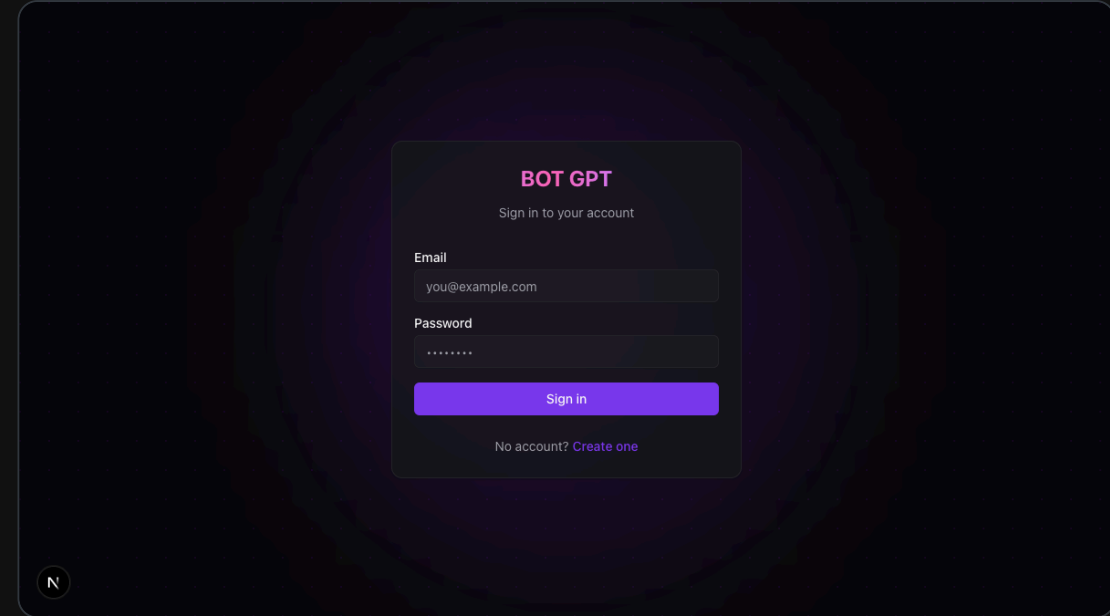


Stateless — no session store needed

Auth: JWT Flow

```
POST /auth/register { email, password }
├─ bcrypt.hash(password, 10)
├─ INSERT INTO users
│   409 ConflictException if email exists
└─ jwt.sign({ sub: userId, email })
    → { access_token }
```

```
POST /auth/login { email, password }
├─ SELECT user WHERE email = ?
├─ bcrypt.compare(password, hash)
└─ 401 UnauthorizedException if wrong
```

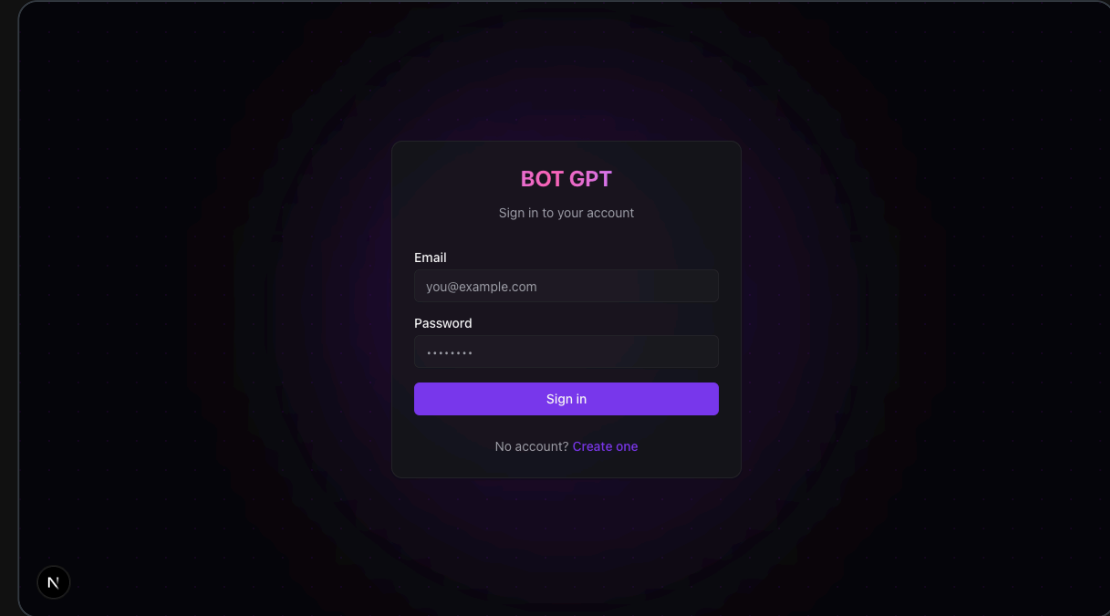


Stateless — no session store needed

Auth: JWT Flow

```
POST /auth/register { email, password }
├─ bcrypt.hash(password, 10)
├─ INSERT INTO users
│   409 ConflictException if email exists
└─ jwt.sign({ sub: userId, email })
    → { access_token }
```

```
POST /auth/login { email, password }
├─ SELECT user WHERE email = ?
├─ bcrypt.compare(password, hash)
│   401 UnauthorizedException if wrong
└─ jwt.sign({ sub: userId, email })
    → { access_token }
```



Stateless — no session store needed

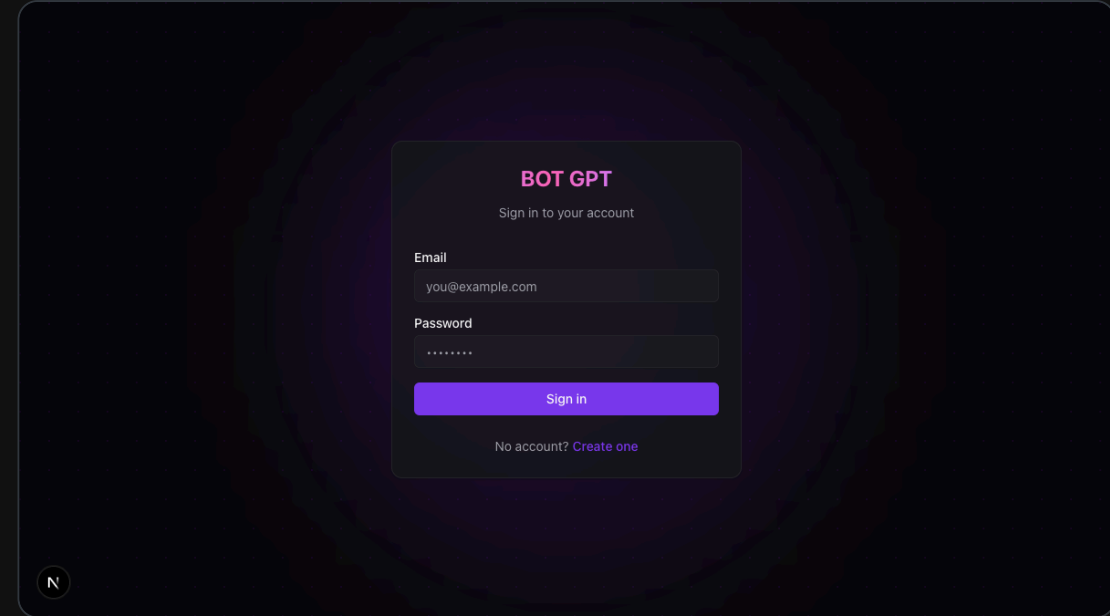
Auth: JWT Flow

```
POST /auth/register { email, password }
├─ bcrypt.hash(password, 10)
├─ INSERT INTO users
│   409 ConflictException if email exists
└─ jwt.sign({ sub: userId, email })
    → { access_token }
```

```
POST /auth/login { email, password }
├─ SELECT user WHERE email = ?
├─ bcrypt.compare(password, hash)
│   401 UnauthorizedException if wrong
└─ jwt.sign({ sub: userId, email })
    → { access_token }
```

Every route:

```
Authorization: Bearer <token>
└─ JwtAuthGuard → req.user = { id, email }
    401 if missing / expired / tampered
```



Stateless — no session store needed

Conversations: Create Flow

Each conversation has two key settings chosen at creation:

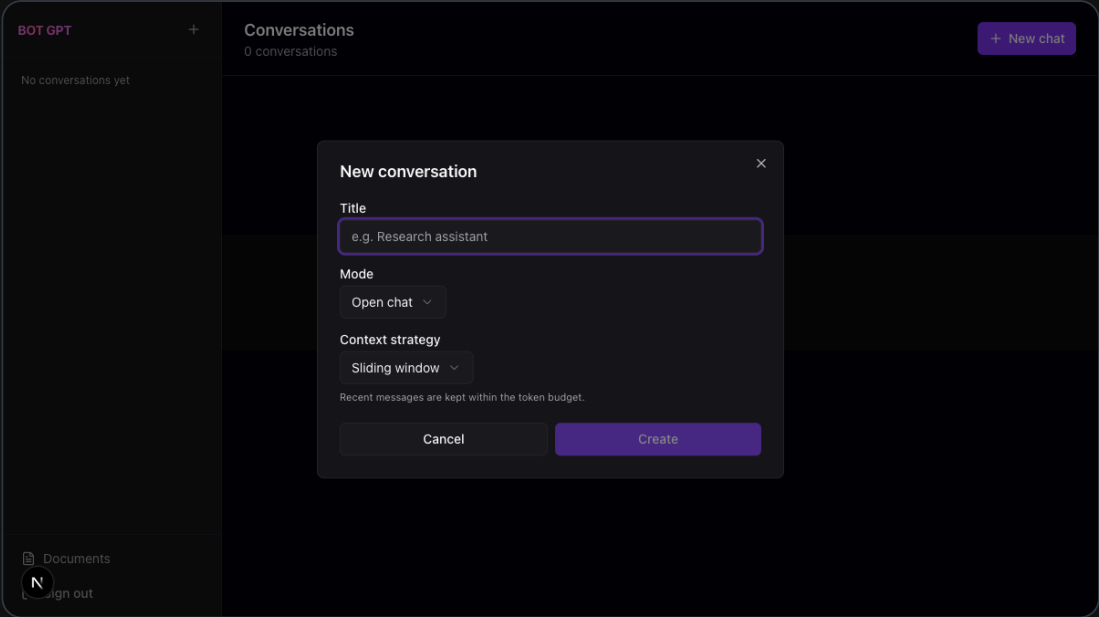
Mode

- Open chat** — pure LLM conversation, no documents.
- RAG** — answers grounded in uploaded PDFs only.

Context Strategy

- Sliding window** — newest messages kept within token budget. Zero extra API calls.
- Summarization** — older turns LLM-compressed into a paragraph. Preserves semantic context.

```
POST /conversations
{ mode, title, context_strategy }
→ stored in conversations table
→ cache.del('convlist:${userId}') ← invalidate
```

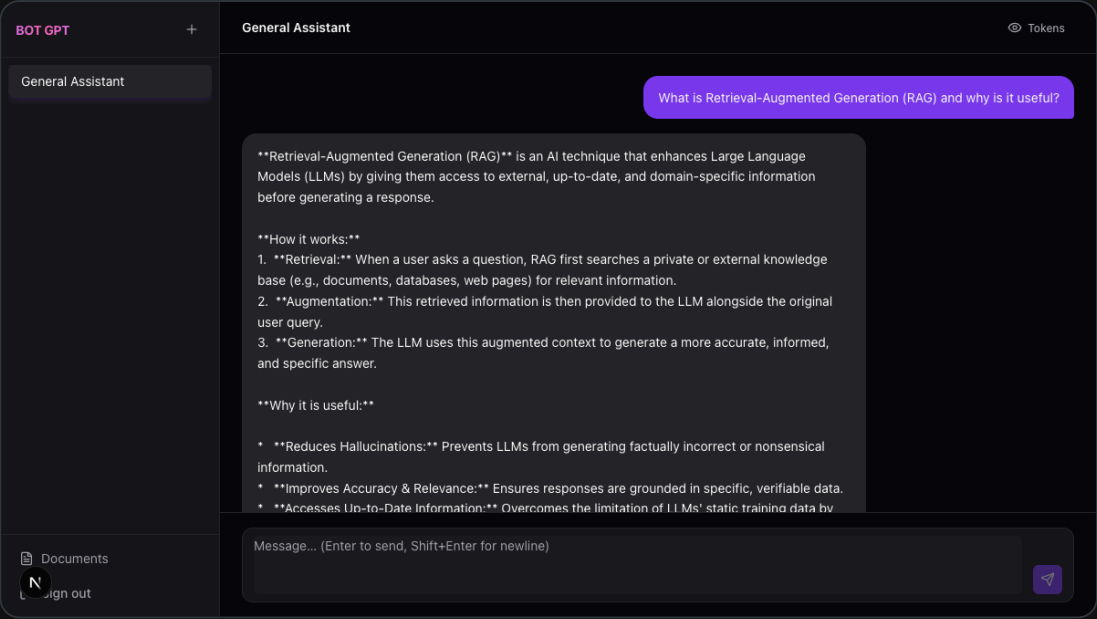


LLM Streaming via SSE

```
POST /conversations/:id/messages
|
|  └─ getDailyTokensUsed(userId) ← sync DB query
|      429 JSON if ≥ 100 000 tokens/day
|      (before SSE headers – still plain JSON)
|
|  └─ res.setHeader('Content-Type',
|      'text/event-stream')
|
|  └─ for await (token of llm.stream(...)) {
|      res.write(`data: {"token":"..."}\n\n`)
|      }
|
|  └─ res.write(`data: {"done":true,
|      "userTokens":12,
|      "assistantTokens":87,
|      "sources":[...]} \n\n`)
|
|  res.end()
```

Simulated token stream:

RAGisanAItechniquethataugmentsLLMswithexternalknowledge.█



Real Gemini 2.5 Flash response

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)

messages = [msg1 msg2 msg3 msg4 msg5]
           oldest  ──────────> newest
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)
```

```
messages = [msg1 msg2 msg3 msg4 msg5]  
           oldest  → newest
```

```
walk newest → oldest:
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)
```

```
messages = [msg1 msg2 msg3 msg4 msg5]  
           oldest  ──────────> newest
```

```
walk newest → oldest:
```

```
msg5 80 t total=80 ✓ add
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)
```

```
messages = [msg1 msg2 msg3 msg4 msg5]  
           oldest  → newest
```

```
walk newest → oldest:
```

```
msg5  80 t  total=80  ✓ add
```

```
msg4 120 t  total=200 ✓ add
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)

messages = [msg1 msg2 msg3 msg4 msg5]
           oldest  ──────────> newest

walk newest → oldest:
  msg5  80 t  total=80    ✓ add
  msg4 120 t  total=200   ✓ add
  msg3 7400 t total=7600  ✓ add
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)

messages = [msg1 msg2 msg3 msg4 msg5]
           oldest      newest

walk newest → oldest:
  msg5  80 t   total=80   ✓ add
  msg4 120 t   total=200  ✓ add
  msg3 7400 t total=7600  ✓ add
  msg2  900 t total=8500 ✗ exceeds budget → stop
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)

messages = [msg1 msg2 msg3 msg4 msg5]
           oldest      ──────────> newest

walk newest → oldest:
  msg5  80 t   total=80   ✓ add
  msg4 120 t   total=200  ✓ add
  msg3 7400 t total=7600  ✓ add
  msg2  900 t   total=8500 ✗ exceeds budget → stop

window = [msg3, msg4, msg5] ← chronological order
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Sliding Window

Default — zero extra API calls. Always keeps the most recent context.

```
budget = 8 000 tokens (open) | 6 000 (RAG)
```

```
messages = [msg1 msg2 msg3 msg4 msg5]
           oldest   ──────────> newest
```

```
walk newest → oldest:
```

```
msg5  80 t   total=80   ✓ add
msg4  120 t  total=200  ✓ add
msg3  7400 t total=7600 ✓ add
msg2   900 t total=8500 ✗ exceeds budget → stop
```

```
window = [msg3, msg4, msg5] ← chronological order
```

```
Edge case: msg5 alone > budget?
```

```
→ still include it (minimum-1 guarantee)
```

```
→ conversation continues, just no history
```

Why newest → oldest?

We want to guarantee the most recent turn is always present. Walking from oldest→newest would drop it under budget pressure.

Token estimation

`Math.ceil(text.length / 4)` — synchronous, zero latency. Stored on each message at save time.

Budget split

RAG uses 6k (not 8k) to leave headroom for the retrieved chunk context block injected into the final message.

Context Strategy: Summarization

For long conversations — preserves semantic content of old turns at the cost of one extra LLM call.



Context Strategy: Summarization

For long conversations — preserves semantic content of old turns at the cost of one extra LLM call.

```
Step 1  first-pass window
firstPass = buildContextWindow(messages, budget=8000)
older     = messages before firstPass → [msg1 ... msg94]
```

Context Strategy: Summarization

For long conversations — preserves semantic content of old turns at the cost of one extra LLM call.

```
Step 1  first-pass window
    firstPass = buildContextWindow(messages, budget=8000)
    older     = messages before firstPass → [msg1 ... msg94]

Step 2  summarize older slice  (1 LLM call)
    summary      = await aiService.summarizeHistory(older)
    summaryTokens = estimateTokens(summary)    // e.g. 350
```


Context Strategy: Summarization

For long conversations — preserves semantic content of old turns at the cost of one extra LLM call.

Step 1 first-pass window

```
firstPass = buildContextWindow(messages, budget=8000)
older     = messages before firstPass → [msg1 ... msg94]
```

Step 2 summarize older slice (1 LLM call)

```
summary      = await aiService.summarizeHistory(older)
summaryTokens = estimateTokens(summary)    // e.g. 350
```

Step 3 rebuild recent window accounting for summary

```
recent = buildContextWindow(messages, 8000 - 350)
← summary tokens reserved so total never overruns budget
```

Context Strategy: Summarization

For long conversations — preserves semantic content of old turns at the cost of one extra LLM call.

```
Step 1  first-pass window
    firstPass = buildContextWindow(messages, budget=8000)
    older      = messages before firstPass → [msg1 ... msg94]
```

```
Step 2  summarize older slice  (1 LLM call)
    summary      = await aiService.summarizeHistory(older)
    summaryTokens = estimateTokens(summary)    // e.g. 350
```

```
Step 3  rebuild recent window accounting for summary
    recent = buildContextWindow(messages, 8000 - 350)
    ← summary tokens reserved so total never overruns budget
```

```
Step 4  final context to Gemini:
[
  SystemMessage("You are BOT GPT..."),
  AssistantMessage("[Earlier conversation summary]: ..."),
  ...recent messages...,
  HumanMessage(current query)
]
```

Context Strategy: Summarization

For long conversations — preserves semantic content of old turns at the cost of one extra LLM call.

```
Step 1  first-pass window
    firstPass = buildContextWindow(messages, budget=8000)
    older      = messages before firstPass → [msg1 ... msg94]

Step 2  summarize older slice  (1 LLM call)
    summary      = await aiService.summarizeHistory(older)
    summaryTokens = estimateTokens(summary)    // e.g. 350

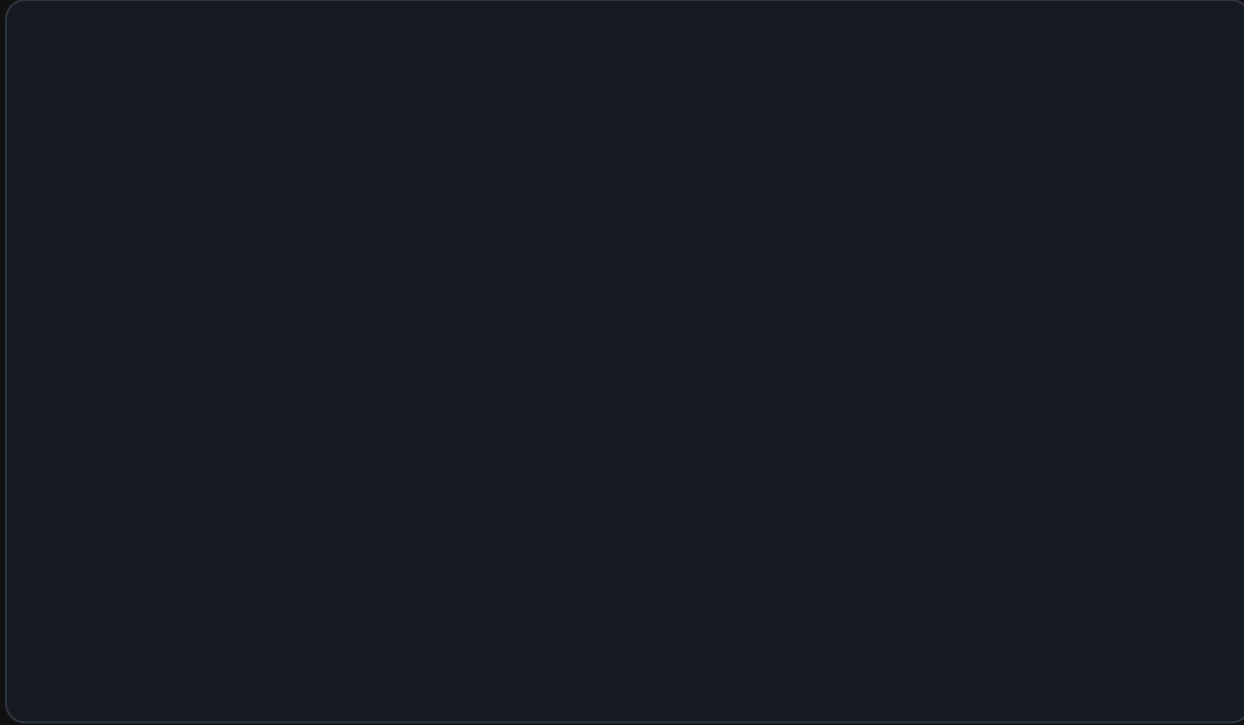
Step 3  rebuild recent window accounting for summary
    recent = buildContextWindow(messages, 8000 - 350)
    ← summary tokens reserved so total never overruns budget

Step 4  final context to Gemini:
[
    SystemMessage("You are BOT GPT..."),
    AssistantMessage("[Earlier conversation summary]: ..."),
    ...recent messages...,
    HumanMessage(current query)
]
```

Bug fixed this session: original code built recent window at full budget, *then* prepended summary — could silently exceed the budget. Fixed by deducting summary tokens before the second buildContextWindow call.

RAG Pipeline

Upload path



Query path



Upload path

POST /documents/:convId

Query path

Upload path

POST /documents/:convId



Upload → Cloudflare R2

Query path

Upload path

POST /documents/:convId



Upload → Cloudflare R2



RecursiveCharacterTextSplitter
chunkSize:1000 overlap:200

Query path

Upload path

POST /documents/:convId



Upload → Cloudflare R2



RecursiveCharacterTextSplitter
chunkSize:1000 overlap:200



GoogleGenerativeAIEmbeddings
→ VECTOR(768) per chunk

Query path

Upload path



Query path



RAG Pipeline

Upload path

```
POST /documents/:convId
```



```
Upload → Cloudflare R2
```



```
RecursiveCharacterTextSplitter  
chunkSize:1000 overlap:200
```



```
GoogleGenerativeAIEmbeddings  
→ VECTOR(768) per chunk
```



```
PGVectorStore.addDocuments()  
metadata: {doc_id, conv_id, filename}
```

Query path

```
POST /conversations/:id/messages
```

RAG Pipeline

Upload path

POST /documents/:convId



Upload → Cloudflare R2



```
RecursiveCharacterTextSplitter  
chunkSize:1000 overlap:200
```



```
GoogleGenerativeAIEmbeddings  
→ VECTOR(768) per chunk
```



```
PGVectorStore.addDocuments()  
metadata: {doc_id, conv_id, filename}
```

Query path

POST /conversations/:id/messages



```
similaritySearch(query, topK=5,  
filter: { conversation_id })
```

RAG Pipeline

Upload path

POST /documents/:convId



Upload → Cloudflare R2



RecursiveCharacterTextSplitter
chunkSize:1000 overlap:200



GoogleGenerativeAIEmbeddings
→ VECTOR(768) per chunk



```
PGVectorStore.addDocuments()  
metadata: {doc_id, conv_id, filename}
```

Query path

POST /conversations/:id/messages



```
similaritySearch(query, topK=5,  
filter: { conversation_id })
```



Build context block:
"Context:\nchunk1\n---\nchunk2\n...
Using only the context above, answer: {q}"

RAG Pipeline

Upload path

POST /documents/:convId

↓ Upload → Cloudflare R2

↓ RecursiveCharacterTextSplitter
chunkSize:1000 overlap:200

↓ GoogleGenerativeAIEmbeddings
→ VECTOR(768) per chunk

↓ PGVectorStore.addDocuments()
metadata: {doc_id, conv_id, filename}

Query path

POST /conversations/:id/messages

↓ similaritySearch(query, topK=5,
filter: { conversation_id })

↓ Build context block:
"Context:\nchunk1\n---\nchunk2\n...
Using only the context above, answer: {q}"

↓ withRetry() → llm.stream(messages)

RAG Pipeline

Upload path

POST /documents/:convId



Upload → Cloudflare R2



RecursiveCharacterTextSplitter
chunkSize:1000 overlap:200



GoogleGenerativeAIEmbeddings
→ VECTOR(768) per chunk



PGVectorStore.addDocuments()
metadata: {doc_id, conv_id, filename}

Query path

POST /conversations/:id/messages



similaritySearch(query, topK=5,
filter: { conversation_id })



Build context block:
"Context:\nchunk1\n---\nchunk2\n...
Using only the context above, answer: {q}"



withRetry(() → llm.stream(messages))



done event: sources[{filename, content}]
→ frontend filename badges

RAG Pipeline

Upload path

POST /documents/:convId

↓ Upload → Cloudflare R2

↓ RecursiveCharacterTextSplitter
chunkSize:1000 overlap:200

↓ GoogleGenerativeAIEmbeddings
→ VECTOR(768) per chunk

↓ PGVectorStore.addDocuments()
metadata: {doc_id, conv_id, filename}

Query path

POST /conversations/:id/messages

↓ similaritySearch(query, topK=5,
filter: { conversation_id })

↓ Build context block:
"Context:\nchunk1\n---\nchunk2\n...
Using only the context above, answer: {q}"

↓ withRetry(() → llm.stream(messages))

↓ done event: sources[{filename, content}]
→ frontend filename badges

Key: `filter: { conversation_id }` scopes vector search to this conversation's documents only — no cross-contamination.

Documents Management

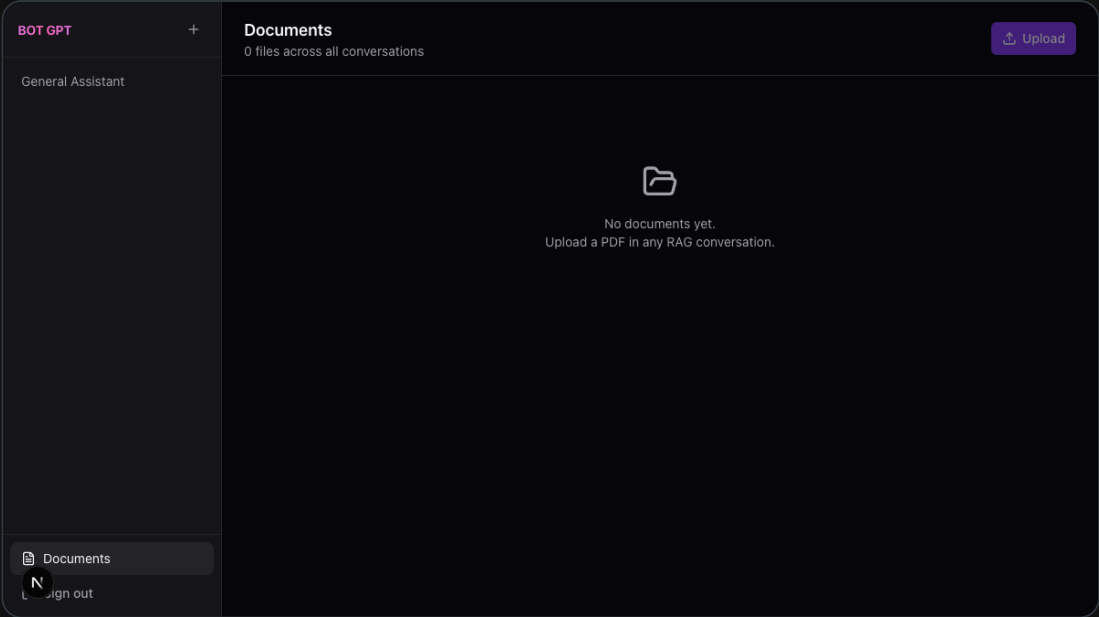
```
POST /documents/:convId
  multipart/form-data { file: PDF, max 10 MB }
  → status: 'processing' (returned immediately)
  → embedding runs synchronously
  → status: 'ready' (or 'failed' on error)

GET /documents
  returns all docs for authenticated user
  across all conversations

DELETE /documents/:id
  1. delete row from documents table
  2. vectorStore.delete({
      filter: { document_id }
    }) ← removes all chunks from pgvector
```

Why Cloudflare R2?

S3-compatible API — zero egress fees vs AWS S3. Raw file stored for potential re-processing. Public URL returned and saved in `documents.r2_url`.



Upload button triggers multipart POST

Daily Token Limit

Hard cap of **100 000 tokens / user / UTC day** — enforced before any LLM call or SSE headers.

Why before SSE headers?

Once `Content-Type: text/event-stream` is written, the response is locked — you can only write SSE events, not switch to a JSON error body. The limit check must happen first.

Resets at UTC midnight

`startOfDay.setUTCHours(0,0,0,0)` — consistent boundary regardless of user timezone.

Configured via env

`DAILY_TOKEN_LIMIT=100000` — defaults to 100k if not set. Adjustable per environment.

Daily Token Limit

Hard cap of **100 000 tokens / user / UTC day** — enforced before any LLM call or SSE headers.

```
getDailyTokensUsed(userId):  
    SELECT COALESCE(SUM(msg.token_count), 0)  
    FROM   messages msg  
    JOIN   conversations conv  
           ON conv.id = msg.conversation_id  
    WHERE  conv.user_id = :userId  
    AND    msg.created_at >= :startOfUTCDay
```

Why before SSE headers?

Once `Content-Type: text/event-stream` is written, the response is locked — you can only write SSE events, not switch to a JSON error body. The limit check must happen first.

Resets at UTC midnight

`startOfDay.setUTCHours(0,0,0,0)` — consistent boundary regardless of user timezone.

Configured via env

`DAILY_TOKEN_LIMIT=100000` — defaults to 100k if not set. Adjustable per environment.

Daily Token Limit

Hard cap of **100 000 tokens / user / UTC day** — enforced before any LLM call or SSE headers.

```
getDailyTokensUsed(userId):
    SELECT COALESCE(SUM(msg.token_count),0)
    FROM   messages msg
    JOIN   conversations conv
          ON conv.id = msg.conversation_id
    WHERE  conv.user_id = :userId
    AND    msg.created_at >= :startOfUTCDay

sendMessage():
    const dailyUsed = await getDailyTokensUsed(...)
    if (dailyUsed >= 100_000) {
        return res.status(429).json({
            error: "Daily limit of 100,000 tokens
                  reached. Resets at midnight UTC."
        })    ← plain JSON, not an SSE event
    }
    // proceed: set SSE headers, stream...
```

Why before SSE headers?

Once `Content-Type: text/event-stream` is written, the response is locked — you can only write SSE events, not switch to a JSON error body. The limit check must happen first.

Resets at UTC midnight

`startOfDay.setUTCHours(0,0,0,0)` — consistent boundary regardless of user timezone.

Configured via env

`DAILY_TOKEN_LIMIT=100000` — defaults to 100k if not set. Adjustable per environment.

Daily Token Limit

Hard cap of **100 000 tokens / user / UTC day** — enforced before any LLM call or SSE headers.

```
getDailyTokensUsed(userId):
    SELECT COALESCE(SUM(msg.token_count),0)
    FROM   messages msg
    JOIN   conversations conv
           ON conv.id = msg.conversation_id
    WHERE  conv.user_id = :userId
    AND    msg.created_at >= :startOfUTCDay

sendMessage():
    const dailyUsed = await getDailyTokensUsed(...)
    if (dailyUsed >= 100_000) {
        return res.status(429).json({
            error: "Daily limit of 100,000 tokens
                   reached. Resets at midnight UTC."
        })    ← plain JSON, not an SSE event
    }
    // proceed: set SSE headers, stream...

After stream completes:
    saveMessage(id, 'user',      content, userTokens)
    saveMessage(id, 'assistant', full,   assistantTokens)
    convRepo.increment(id, 'token_count', total)
```

Why before SSE headers?

Once `Content-Type: text/event-stream` is written, the response is locked — you can only write SSE events, not switch to a JSON error body. The limit check must happen first.

Resets at UTC midnight

`startOfDay.setUTCHours(0,0,0,0)` — consistent boundary regardless of user timezone.

Configured via env

`DAILY_TOKEN_LIMIT=100000` — defaults to 100k if not set. Adjustable per environment.

LLM Retry: Exponential Backoff

```
private async withRetry<T>(  
  fn: () => Promise<T>,  
  maxAttempts = 3,  
) : Promise<T> {  
  let delay = 1000;  
  let lastError: unknown;  
  for (let i = 1; i <= maxAttempts; i++) {  
    try {  
      return await fn();  
    } catch (err) {  
      lastError = err;  
      if (i < maxAttempts) {  
        await sleep(delay);  
        delay *= 2; // 1s → 2s → 4s  
      }  
    }  
  }  
  throw lastError; // bubble up after 3 fails  
}  
  
// Usage:  
const stream = await this.withRetry(  
  () => this.llm.stream(messages)  
);
```

Tested with Jest

Mock `setTimeout` to run instantly, make `llm.stream` fail twice then succeed. Assert tokens arrive on attempt 3 and `mockStream` called exactly 3 times.

LLM Retry: Exponential Backoff

```
private async withRetry<T>(  
  fn: () => Promise<T>,  
  maxAttempts = 3,  
) : Promise<T> {  
  let delay = 1000;  
  let lastError: unknown;  
  for (let i = 1; i <= maxAttempts; i++) {  
    try {  
      return await fn();  
    } catch (err) {  
      lastError = err;  
      if (i < maxAttempts) {  
        await sleep(delay);  
        delay *= 2; // 1s → 2s → 4s  
      }  
    }  
  }  
  throw lastError; // bubble up after 3 fails  
}  
  
// Usage:  
const stream = await this.withRetry(  
  () => this.llm.stream(messages)  
);
```

```
Attempt 1 → Gemini  
503 error  
sleep 1 000 ms
```

Tested with Jest

Mock `setTimeout` to run instantly, make `llm.stream` fail twice then succeed. Assert tokens arrive on attempt 3 and `mockStream` called exactly 3 times.

LLM Retry: Exponential Backoff

```
private async withRetry<T>(  
  fn: () => Promise<T>,  
  maxAttempts = 3,  
) : Promise<T> {  
  let delay = 1000;  
  let lastError: unknown;  
  for (let i = 1; i <= maxAttempts; i++) {  
    try {  
      return await fn();  
    } catch (err) {  
      lastError = err;  
      if (i < maxAttempts) {  
        await sleep(delay);  
        delay *= 2; // 1s → 2s → 4s  
      }  
    }  
  }  
  throw lastError; // bubble up after 3 fails  
}  
  
// Usage:  
const stream = await this.withRetry(  
  () => this.llm.stream(messages)  
);
```

Attempt 1 → Gemini
503 error
sleep 1 000 ms

Attempt 2 → Gemini
503 error
sleep 2 000 ms

Tested with Jest

Mock `setTimeout` to run instantly, make `llm.stream` fail twice then succeed. Assert tokens arrive on attempt 3 and `mockStream` called exactly 3 times.

LLM Retry: Exponential Backoff

```
private async withRetry<T>(  
  fn: () => Promise<T>,  
  maxAttempts = 3,  
) : Promise<T> {  
  let delay = 1000;  
  let lastError: unknown;  
  for (let i = 1; i <= maxAttempts; i++) {  
    try {  
      return await fn();  
    } catch (err) {  
      lastError = err;  
      if (i < maxAttempts) {  
        await sleep(delay);  
        delay *= 2; // 1s → 2s → 4s  
      }  
    }  
  }  
  throw lastError; // bubble up after 3 fails  
}  
  
// Usage:  
const stream = await this.withRetry(  
  () => this.llm.stream(messages)  
);
```

Attempt 1 → Gemini
503 error
sleep 1 000 ms

Attempt 2 → Gemini
503 error
sleep 2 000 ms

Attempt 3 → Gemini
200 OK → tokens stream

Tested with Jest

Mock `setTimeout` to run instantly, make `llm.stream` fail twice then succeed. Assert tokens arrive on attempt 3 and `mockStream` called exactly 3 times.

LLM Retry: Exponential Backoff

```
private async withRetry<T>(  
  fn: () => Promise<T>,  
  maxAttempts = 3,  
) : Promise<T> {  
  let delay = 1000;  
  let lastError: unknown;  
  for (let i = 1; i <= maxAttempts; i++) {  
    try {  
      return await fn();  
    } catch (err) {  
      lastError = err;  
      if (i < maxAttempts) {  
        await sleep(delay);  
        delay *= 2; // 1s → 2s → 4s  
      }  
    }  
  }  
  throw lastError; // bubble up after 3 fails  
}  
  
// Usage:  
const stream = await this.withRetry(  
  () => this.llm.stream(messages)  
);
```

Attempt 1 → Gemini
503 error
sleep 1 000 ms

Attempt 2 → Gemini
503 error
sleep 2 000 ms

Attempt 3 → Gemini
200 OK → tokens stream

If all 3 fail:
SSE handler catches error
res.write(`{"error":"LLM unavailable"}`)
res.end()

Tested with Jest

Mock `setTimeout` to run instantly, make `llm.stream` fail twice then succeed. Assert tokens arrive on attempt 3 and `mockStream` called exactly 3 times.

Conversation List Caching

```
NestJS CacheModule config (app.module.ts):
```

```
CacheModule.register({
  isGlobal: true,
  ttl: 60_000    // 60 seconds
})
```

```
@Inject(CACHE_MANAGER) private cache: Cache
```

Why in-memory, not Redis?

Single instance deployment. NestJS `CacheModule` (node-cache internally) adds zero infrastructure. Redis is the right call when running multiple API instances that need a *shared* cache.

What's cached

Only `GET /conversations` — the highest-frequency read. Single conversation fetches (`GET /conversations/:id`) are not cached since they include the full message list and change with every send.

Switching to Redis later

Replace `CacheModule.register()` with `CacheModule.registerAsync()` + `@nestjs/cache-manager` Redis store. Zero changes to service code.

Conversation List Caching

```
async findAll(userId, page, limit) {  
  const key = `convlist:${userId}:${page}:${limit}`
```

```
NestJS CacheModule config (app.module.ts):
```

```
CacheModule.register({  
  isGlobal: true,  
  ttl: 60_000    // 60 seconds  
})
```

```
@Inject(CACHE_MANAGER) private cache: Cache
```

Why in-memory, not Redis?

Single instance deployment. NestJS `CacheModule` (node-cache internally) adds zero infrastructure. Redis is the right call when running multiple API instances that need a *shared* cache.

What's cached

Only `GET /conversations` — the highest-frequency read. Single conversation fetches (`GET /conversations/:id`) are not cached since they include the full message list and change with every send.

Switching to Redis later

Replace `CacheModule.register()` with `CacheModule.registerAsync()` + `@nestjs/cache-manager` Redis store. Zero changes to service code.

Conversation List Caching

```
async findAll(userId, page, limit) {
  const key = `convlist:${userId}:${page}:${limit}`

  const cached = await cache.get(key)
  if (cached) return cached ← cache hit
}
```

```
NestJS CacheModule config (app.module.ts):

CacheModule.register({
  isGlobal: true,
  ttl: 60_000    // 60 seconds
})

@Inject(CACHE_MANAGER) private cache: Cache
```

Why in-memory, not Redis?

Single instance deployment. NestJS `CacheModule` (node-cache internally) adds zero infrastructure. Redis is the right call when running multiple API instances that need a *shared* cache.

What's cached

Only `GET /conversations` — the highest-frequency read. Single conversation fetches (`GET /conversations/:id`) are not cached since they include the full message list and change with every send.

Switching to Redis later

Replace `CacheModule.register()` with `CacheModule.registerAsync()` + `@nestjs/cache-manager` Redis store. Zero changes to service code.

Conversation List Caching

```
async findAll(userId, page, limit) {
  const key = `convlist:${userId}:${page}:${limit}`

  const cached = await cache.get(key)
  if (cached) return cached ← cache hit

  const result = await convRepo.find(...)
  await cache.set(key, result, ttl=60_000)
  return result
}
```

```
NestJS CacheModule config (app.module.ts):

CacheModule.register({
  isGlobal: true,
  ttl: 60_000 // 60 seconds
})

@Inject(CACHE_MANAGER) private cache: Cache
```

Why in-memory, not Redis?

Single instance deployment. NestJS `CacheModule` (node-cache internally) adds zero infrastructure. Redis is the right call when running multiple API instances that need a *shared* cache.

What's cached

Only `GET /conversations` — the highest-frequency read. Single conversation fetches (`GET /conversations/:id`) are not cached since they include the full message list and change with every send.

Switching to Redis later

Replace `CacheModule.register()` with `CacheModule.registerAsync()` + `@nestjs/cache-manager` Redis store. Zero changes to service code.

Conversation List Caching

```
async findAll(userId, page, limit) {
  const key = `convlist:${userId}:${page}:${limit}`

  const cached = await cache.get(key)
  if (cached) return cached ← cache hit

  const result = await convRepo.find(...)
  await cache.set(key, result, ttl=60_000)
  return result
}

// Invalidation on mutations:
create() → cache.del(`convlist:${userId}`)
remove() → cache.del(`convlist:${userId}`)
          ← wildcard-style: all pages cleared
```

```
NestJS CacheModule config (app.module.ts):

CacheModule.register({
  isGlobal: true,
  ttl: 60_000    // 60 seconds
})

@Inject(CACHE_MANAGER) private cache: Cache
```

Why in-memory, not Redis?

Single instance deployment. NestJS `CacheModule` (node-cache internally) adds zero infrastructure. Redis is the right call when running multiple API instances that need a *shared* cache.

What's cached

Only `GET /conversations` — the highest-frequency read. Single conversation fetches (`GET /conversations/:id`) are not cached since they include the full message list and change with every send.

Switching to Redis later

Replace `CacheModule.register()` with `CacheModule.registerAsync()` + `@nestjs/cache-manager` Redis store. Zero changes to service code.

Deployment: Docker + CI

Multi-Stage Dockerfile

```
# Stage 1: deps - layer-cached
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

# Stage 2: builder
FROM node:20-alpine AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build          # → dist/

# Stage 3: runner (minimal)
FROM node:20-alpine AS runner
ENV NODE_ENV=production
RUN addgroup --system --gid 1001 nodejs \
  && adduser --system --uid 1001 nestjs
COPY --from=builder /app/dist ./dist
COPY --from=deps /app/node_modules ./node_modules
COPY package*.json ./
USER nestjs                # non-root
EXPOSE 3001
CMD ["node", "dist/main"]
```

GitHub Actions CI

```
on: [push, pull_request]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run lint

  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg17
        env:
          POSTGRES_DB: botgpt_test
        options: >-
          --health-cmd pg_isready
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
```

Why non-root user?

If the container is compromised, `uid 1001` cannot write to system directories or escalate privileges. Security best practice for production images.

Testing: TDD Throughout

Every feature: RED test first → implementation → GREEN. 40 tests, 5 suites, <2 seconds.

Suite	Key scenarios	Tests
auth.service.spec.ts	Register (duplicate→409), login (bad pw→401), JWT issued on success	5
conversations.service.spec.ts	Create with default strategy, pagination, ownership 403/404, sliding window budget (incl. oversized message edge case), daily token query	14
documents.service.spec.ts	Upload + embed flow, list by user, delete + pgvector chunk cleanup	5
ai.service.spec.ts	estimateTokens, streamOpenChat, streamRagChat context injection, retry (2 fails then success; all 3 fail), summarizeHistory (3 cases), retrieveRelevantDocs	16

All external deps (Gemini, pgvector, TypeORM repos) are Jest-mocked. Retry tests mock `setTimeout` to run instantly — no real waits. Tests run with zero network or DB required.

Where it Breaks at Scale

```
graph TD
    Users --> API_layer
    subgraph API_layer [API layer]
        direction TB
        A1[← stateless, scales horizontally behind any LB]
    end
    API_layer --> PostgreSQL
    subgraph PostgreSQL [PostgreSQL]
        direction TB
        P1[← single writer, ~10k concurrent writes before it struggles]
    end
    PostgreSQL --> Gemini_API_rate_limits
    subgraph Gemini_API_rate_limits [Gemini API rate limits]
        direction TB
        G1[← shared quota across all users]
    end
```

Users → **API layer** ← stateless, scales horizontally behind any LB

↓

PostgreSQL ← single writer, ~10k concurrent writes before it struggles

↓

Gemini API rate limits ← shared quota across all users

DB bottleneck

Read replicas for `GET /conversations` + message history.
PgBouncer for connection pooling.
Partition `doc_embeddings` by `conversation_id`.
Shard `messages` by `conversation_id` at extreme scale.

LLM cost

Redis cache for repeated RAG queries (same question + same docs = same answer).
BullMQ queue for document embedding — async, retryable, off the request path.

Caching

Swap in-memory `CacheModule` for Redis store.
Zero service code changes — just config swap.
Enables shared cache across multiple API instances.

Everything Built

- ✓ JWT auth — register + login
- ✓ Streaming chat via SSE
- ✓ RAG — pgvector + Gemini embeddings
- ✓ Sliding window context strategy
- ✓ Summarization context strategy
- ✓ Daily token limit (100k / day)
- ✓ Exponential backoff retry (3×)
- ✓ In-memory conversation list cache
- ✓ Multi-stage Docker (non-root)
- ✓ GitHub Actions CI (lint + test)
- ✓ Next.js 15 frontend + shadcn/ui
- ✓ 40 unit tests · TDD throughout

NestJS

PostgreSQL + pgvector

Gemini 2.5 Flash

LangChain

Next.js 15

Docker

GitHub Actions